구성

NetApp Solutions

NetApp
May 17, 2022

This PDF was generated from https://docs.netapp.com/ko-kr/netapp-solutions/containers/rh-os-n_use_case_multitenancy_configuration_prerequisites.html on May 17, 2022. Always check docs.netapp.com for the latest.

목차

구성 1
- 구성 1
- 구성: 클러스터 관리 작업 2
- 구성: 스토리지 관리 작업 6
- 검증 9
- 확장: 더 많은 프로젝트 추가 14

구성

모든 멀티테넌트 솔루션의 경우 사용자는 필요한 것보다 더 많은 클러스터 리소스에 액세스할 수 없습니다. 따라서 다중 테넌시 구성의 일부로 구성할 전체 리소스 세트는 클러스터 관리자, 스토리지 관리자 및 각 프로젝트를 작업하는 개발자가 구성합니다.

다음 표에는 여러 사용자가 수행해야 하는 여러 작업이 정리되어 있습니다.

역할	작업
* 클러스터 관리자 *	다양한 애플리케이션 또는 워크로드를 위한 프로젝트를 생성합니다
	storage-admin에 대한 ClusterRoles 및 RoleBindings를 생성합니다
	특정 프로젝트에 대한 액세스를 할당하는 개발자를 위한 역할 및 RoleBindings를 만듭니다
	[선택 사항] 특정 노드에서 Pod를 예약하도록 프로젝트를 구성합니다
* 스토리지 - 관리자 *	NetApp ONTAP에서 SVM을 생성합니다
	Trident 백엔드를 생성합니다
	StorageClasses를 생성합니다
	스토리지 리소스 할당량을 생성합니다
개발자 *	할당된 프로젝트에서 PVC 또는 POD를 생성하거나 패치하기 위한 액세스를 검증합니다
	다른 프로젝트에서 PVC 또는 POD를 생성하거나 패치하기 위한 액세스를 검증합니다
	프로젝트, 리소스 할당량 및 StorageClasses를 보거나 편집하려면 액세스 유효성을 검사합니다

"다음: 필수 구성 요소."

구성

필수 구성 요소

- NetApp ONTAP 클러스터
- Red Hat OpenShift 클러스터
- 클러스터에 설치된 Trident
- tridentctl 및 OC 도구가 설치되고 $PATH에 추가된 관리 워크스테이션
- ONTAP에 대한 관리자 액세스
- OpenShift 클러스터에 대한 클러스터 관리자 액세스
- 클러스터가 ID 공급자와 통합됩니다

- ID 공급자는 서로 다른 팀의 사용자를 효율적으로 구분하도록 구성되어 있습니다

"다음: 클러스터 관리자 작업."

구성: 클러스터 관리 작업

Red Hat OpenShift cluster-admin은 다음과 같은 작업을 수행합니다.

1. Red Hat OpenShift 클러스터에 cluster-admin으로 로그인합니다.
2. 서로 다른 프로젝트에 해당하는 두 개의 프로젝트를 작성합니다.

```
oc create namespace project-1
oc create namespace project-2
```

3. PROJECT-1의 개발자 역할을 만듭니다.

```
cat << EOF | oc create -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: project-1
  name: developer-project-1
rules:
  - verbs:
      - '*'
    apiGroups:
      - apps
      - batch
      - autoscaling
      - extensions
      - networking.k8s.io
      - policy
      - apps.openshift.io
      - build.openshift.io
      - image.openshift.io
      - ingress.operator.openshift.io
      - route.openshift.io
      - snapshot.storage.k8s.io
      - template.openshift.io
    resources:
      - '*'
  - verbs:
      - '*'
    apiGroups:
      - ''
```

```
    resources:
      - bindings
      - configmaps
      - endpoints
      - events
      - persistentvolumeclaims
      - pods
      - pods/log
      - pods/attach
      - podtemplates
      - replicationcontrollers
      - services
      - limitranges
      - namespaces
      - componentstatuses
      - nodes
  - verbs:
      - '*'
    apiGroups:
      - trident.netapp.io
    resources:
      - tridentsnapshots
EOF
```

이 섹션에 제공된 역할 정의는 예일 뿐입니다. 개발자 역할은 최종 사용자 요구 사항에 따라 정의되어야 합니다.

1. 마찬가지로 project-2에 대한 개발자 역할을 만듭니다.
2. 모든 OpenShift 및 NetApp 스토리지 리소스는 일반적으로 스토리지 관리자가 관리합니다. 스토리지 관리자를 위한 액세스는 Trident가 설치될 때 생성되는 덴트 운영자 역할에 의해 제어됩니다. 또한 스토리지 관리자는 리소스 할당량에 대한 액세스 권한이 있어야 스토리지 소비 방식을 제어할 수 있습니다.
3. 클러스터의 모든 프로젝트에서 리소스 할당량을 관리하는 역할을 생성하여 스토리지 관리자에게 연결합니다.

```
cat << EOF | oc create -f -
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: resource-quotas-role
rules:
  - verbs:
      - '*'
    apiGroups:
      - ''
    resources:
      - resourcequotas
  - verbs:
      - '*'
    apiGroups:
      - quota.openshift.io
    resources:
      - '*'
EOF
```

4. 클러스터가 조직의 ID 공급자와 통합되었으며 사용자 그룹이 클러스터 그룹과 동기화되었는지 확인합니다. 다음 예제에서는 ID 공급자가 클러스터와 통합되고 사용자 그룹과 동기화되었음을 보여 줍니다.

```
$ oc get groups
NAME                          USERS
ocp-netapp-storage-admins     ocp-netapp-storage-admin
ocp-project-1                 ocp-project-1-user
ocp-project-2                 ocp-project-2-user
```

1. 스토리지 관리자용 ClusterRoleBindings를 구성합니다.

```
cat << EOF | oc create -f -
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: netapp-storage-admin-trident-operator
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-netapp-storage-admins
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: trident-operator
---
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: netapp-storage-admin-resource-quotas-cr
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-netapp-storage-admins
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: resource-quotas-role
EOF
```

스토리지 관리자의 경우 세 가지 역할, 즉 세 가지 운영자 및 리소스 할당량이 바인딩되어야 합니다.

1. Developer-project-1 역할을 project-1의 해당 그룹(OCP-project-1)에 바인딩하는 개발자를 위한 RoleBindings를 만듭니다.

```
cat << EOF | oc create -f -
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: project-1-developer
  namespace: project-1
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-project-1
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: developer-project-1
EOF
```

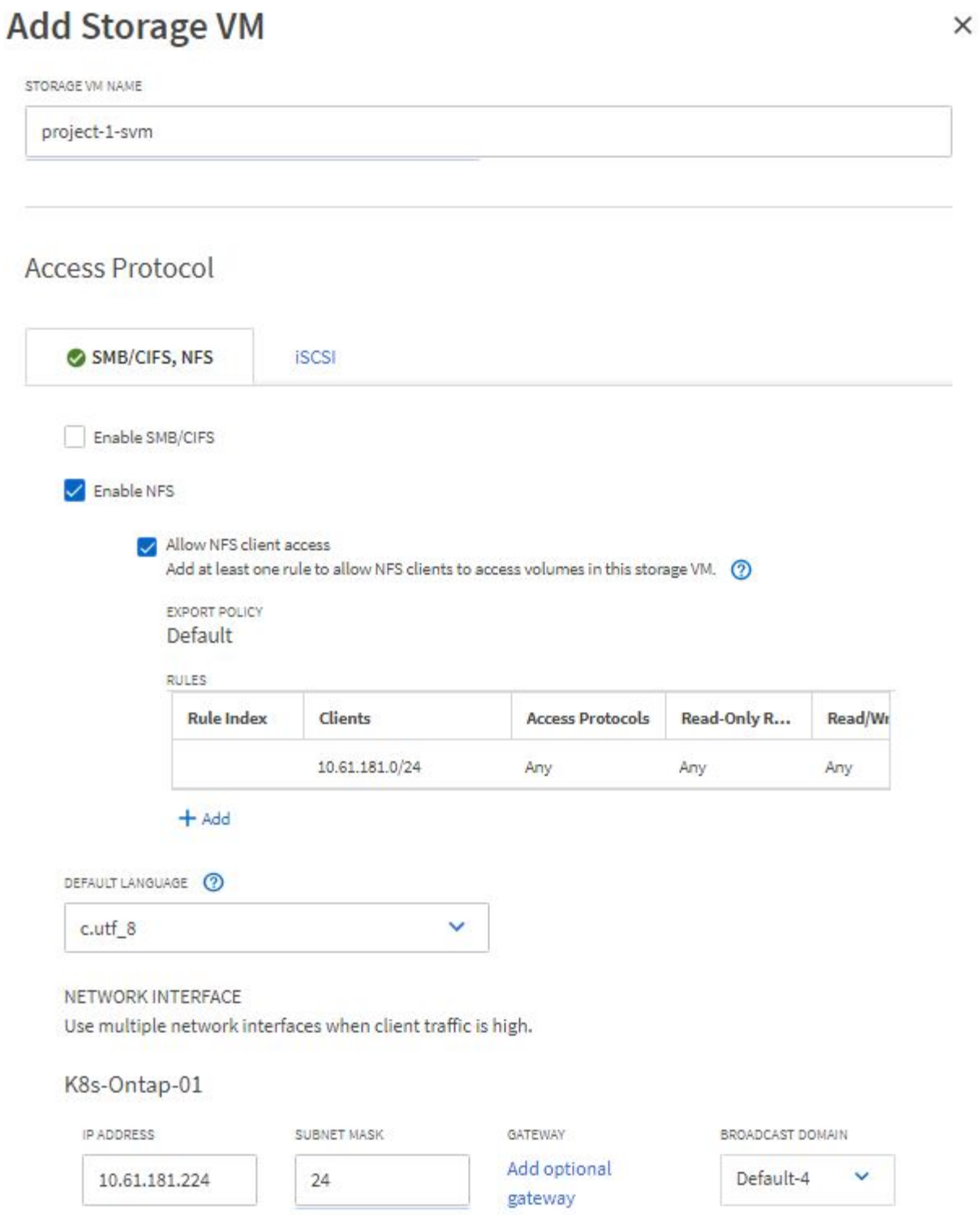
2. 마찬가지로 개발자 역할을 프로젝트 2의 해당 사용자 그룹에 바인딩하는 개발자를 위한 RoleBindings 를 만듭니다.

"다음: 스토리지 관리자 작업."

구성: 스토리지 관리 작업

스토리지 관리자가 다음 리소스를 구성해야 합니다.

1. NetApp ONTAP 클러스터에 admin으로 로그인합니다.
2. Storage > Storage VMs 로 이동하고 Add 를 클릭합니다. 필요한 세부 정보를 제공하여 프로젝트-1과 프로젝트-2에 각각 필요한 SVM 2개를 생성합니다. SVM 및 리소스를 관리하는 vsadmin 계정도 생성합니다.



1. 스토리지 관리자로 Red Hat OpenShift 클러스터에 로그인합니다.
2. 프로젝트-1의 백엔드를 생성하여 프로젝트 전용 SVM에 매핑합니다. ONTAP 클러스터 관리자를 사용하는 대신 SVM의 vsadmin 계정을 사용하여 백엔드를 SVM에 연결하는 것이 좋습니다.

```
cat << EOF | tridentctl -n trident create backend -f
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "nfs_project_1",
    "managementLIF": "172.21.224.210",
    "dataLIF": "10.61.181.224",
    "svm": "project-1-svm",
    "username": "vsadmin",
    "password": "NetApp123"
}
EOF
```

이 예에서는 ONTAP-NAS 드라이버를 사용하고 있습니다. 사용 사례에 따라 백엔드를 생성할 때 적절한 드라이버를 사용하십시오.

트리덴트 프로젝트에 Trident가 설치되어 있다고 가정합니다.

1. 마찬가지로, project-2를 위한 Trident 백엔드를 생성한 다음 project-2 전용 SVM에 매핑합니다.
2. 그런 다음 스토리지 클래스를 생성합니다. storagePools 매개 변수를 설정하여 project-1용 스토리지 클래스를 생성하고 backend 전용 스토리지 풀을 project-1에 사용하도록 구성합니다.

```
cat << EOF | oc create -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: project-1-sc
provisioner: csi.trident.netapp.io
parameters:
  backendType: ontap-nas
  storagePools: "nfs_project_1:.*"
EOF
```

3. 마찬가지로, project-2에 대한 스토리지 클래스를 생성하고 백엔드에서 project-2에 대한 스토리지 풀을 사용하도록 구성합니다.
4. ResourceQuota를 생성하여 다른 프로젝트 전용 스토리지로부터 스토리지를 요청하는 Project-1의 리소스를 제한합니다.

```
cat << EOF | oc create -f -
kind: ResourceQuota
apiVersion: v1
metadata:
  name: project-1-sc-rq
  namespace: project-1
spec:
  hard:
    project-2-sc.storageclass.storage.k8s.io/persistentvolumeclaims: 0
EOF
```

5. 마찬가지로, ResourceQuota를 생성하여 다른 프로젝트 전용 스토리지로부터 스토리지를 요청하는 Project-2의 리소스를 제한합니다.

"다음: 검증."

검증

이전 단계에서 구성한 멀티테넌트 아키텍처를 확인하려면 다음 단계를 수행하십시오.

할당된 프로젝트에서 **PVC** 또는 **POD**를 생성하기 위한 액세스를 검증합니다

1. PROJECT-1의 개발자인 OCP-PROJECT-1-USER로 로그인합니다.
2. 새 프로젝트를 만들려면 액세스 권한을 확인하십시오.

```
oc create ns sub-project-1
```

3. PROJECT-1에 할당된 storageclass를 사용하여 PROJECT-1에서 PVC를 생성한다.

```
cat << EOF | oc create -f -
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-project-1
  namespace: project-1
  annotations:
    trident.netapp.io/reclaimPolicy: Retain
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: project-1-sc
EOF
```

4. PVC와 관련된 PV를 확인한다.

```
oc get pv
```

5. PV 및 해당 볼륨이 NetApp ONTAP의 PROJECT-1 전용 SVM에서 생성되었는지 확인합니다.

```
volume show -vserver project-1-svm
```

6. PROJECT-1에서 POD를 생성하고 이전 단계에서 만든 PVC를 마운트합니다.

```
cat << EOF | oc create -f -
kind: Pod
apiVersion: v1
metadata:
  name: test-pvc-pod
  namespace: project-1
spec:
  volumes:
    - name: test-pvc-project-1
      persistentVolumeClaim:
       claimName: test-pvc-project-1
  containers:
    - name: test-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
      volumeMounts:
        - mountPath: "/usr/share/nginx/html"
          name: test-pvc-project-1
EOF
```

7. POD가 실행 중이고 볼륨이 마운트되어 있는지 확인합니다.

```
oc describe pods test-pvc-pod -n project-1
```

다른 프로젝트에서 **PVC** 또는 **POD**를 생성하거나 다른 프로젝트 전용 리소스를 사용하도록 액세스를 검증합니다

1. PROJECT-1의 개발자인 OCP-PROJECT-1-USER로 로그인합니다.
2. PROJECT-2에 할당된 storageclass를 사용하여 PROJECT-1에서 PVC를 생성한다.

```
cat << EOF | oc create -f -
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-project-1-sc-2
  namespace: project-1
  annotations:
    trident.netapp.io/reclaimPolicy: Retain
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: project-2-sc
EOF
```

3. PROJECT-2에서 PVC를 작성합니다.

```
cat << EOF | oc create -f -
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: test-pvc-project-2-sc-1
  namespace: project-2
  annotations:
    trident.netapp.io/reclaimPolicy: Retain
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: project-1-sc
EOF
```

4. PVC의 TEST-PVC-PROJECT-1-SC-2, TEST-PVC-PROJECT-2-SC-1이 생성되지 않았는지 확인한다.

```
oc get pvc -n project-1
oc get pvc -n project-2
```

5. PROJECT-2에서 POD를 작성합니다.

```
cat << EOF | oc create -f -
kind: Pod
apiVersion: v1
metadata:
  name: test-pvc-pod
  namespace: project-1
spec:
  containers:
    - name: test-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
EOF
```

프로젝트, 리소스 할당량 및 **StorageClasses**를 보고 편집하려면 액세스 권한을 확인합니다

1. PROJECT-1의 개발자인 OCP-PROJECT-1-USER로 로그인합니다.
2. 새 프로젝트를 만들려면 액세스 권한을 선택합니다.

```
oc create ns sub-project-1
```

3. 프로젝트 보기에 대한 액세스 권한 확인

```
oc get ns
```

4. 사용자가 PROJECT-1에서 ResourceQuotas를 보거나 편집할 수 있는지 확인합니다.

```
oc get resourcequotas -n project-1
oc edit resourcequotas project-1-sc-rq -n project-1
```

5. 사용자가 스토리지 시스템을 볼 수 있는 액세스 권한이 있는지 확인합니다.

```
oc get sc
```

6. 스토리지 풀 설명을 위한 액세스를 확인하십시오.
7. 스토리지 풀 편집을 위한 사용자 액세스 권한 검증

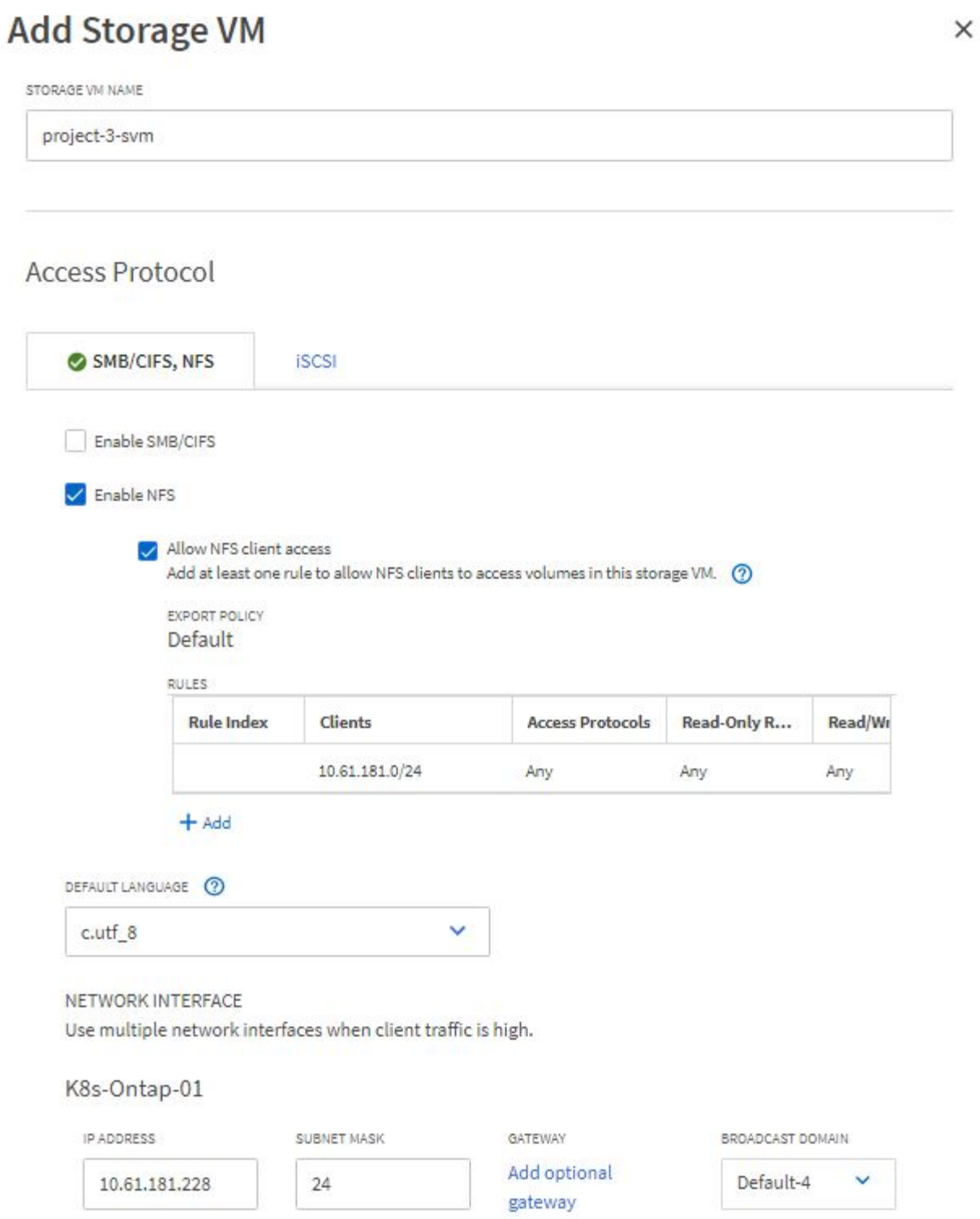
```
oc edit sc project-1-sc
```

"다음: 확장."

확장: 더 많은 프로젝트 추가

멀티 테넌트 구성에서는 스토리지 리소스로 새 프로젝트를 추가하려면 멀티 테넌시가 위반되지 않도록 추가적인 구성이 필요합니다. 멀티 테넌트 클러스터에 프로젝트를 더 추가하려면 다음 단계를 완료하십시오.

1. NetApp ONTAP 클러스터에 스토리지 관리자로 로그인합니다.
2. 스토리지 → 스토리지 VM으로 이동한 후 추가 를 클릭합니다. PROJECT-3 전용 SVM을 새로 생성합니다. SVM 및 리소스를 관리하는 vsadmin 계정도 생성합니다.



1. Red Hat OpenShift 클러스터에 클러스터 관리자로 로그인합니다.
2. 새 프로젝트를 만듭니다.

```
oc create ns project-3
```

3. PROJECT-3의 사용자 그룹이 IDP에서 생성되고 OpenShift 클러스터와 동기화되었는지 확인합니다.

```
oc get groups
```

4. PROJECT-3의 개발자 역할을 만듭니다.

```
cat << EOF | oc create -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: project-3
  name: developer-project-3
rules:
  - verbs:
      - '*'
    apiGroups:
      - apps
      - batch
      - autoscaling
      - extensions
      - networking.k8s.io
      - policy
      - apps.openshift.io
      - build.openshift.io
      - image.openshift.io
      - ingress.operator.openshift.io
      - route.openshift.io
      - snapshot.storage.k8s.io
      - template.openshift.io
    resources:
      - '*'
  - verbs:
      - '*'
    apiGroups:
      - ''
    resources:
      - bindings
      - configmaps
      - endpoints
      - events
      - persistentvolumeclaims
      - pods
      - pods/log
      - pods/attach
      - podtemplates
```

```
      - replicationcontrollers
      - services
      - limitranges
      - namespaces
      - componentstatuses
      - nodes
  - verbs:
      - '*'
    apiGroups:
      - trident.netapp.io
    resources:
      - tridentsnapshots
EOF
```

> 이 섹션에 제공된 역할 정의는 예일 뿐입니다. 개발자 역할은 최종 사용자 요구 사항에 따라 정의되어야 합니다.

1. 개발자를 위한 RoleBinding 만들기 project-3에서 developer-project-3 역할을 프로젝트-3의 해당 그룹(OCP-PROJECT-3)에 바인딩합니다.

```
cat << EOF | oc create -f -
kind: RoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: project-3-developer
  namespace: project-3
subjects:
  - kind: Group
    apiGroup: rbac.authorization.k8s.io
    name: ocp-project-3
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: developer-project-3
EOF
```

2. Red Hat OpenShift 클러스터에 스토리지 관리자로 로그인합니다
3. Trident 백엔드를 생성하여 PROJECT-3 전용 SVM에 매핑합니다. ONTAP 클러스터 관리자를 사용하는 대신 SVM의 vsadmin 계정을 사용하여 백엔드를 SVM에 연결하는 것이 좋습니다.

```
cat << EOF | tridentctl -n trident create backend -f
{
    "version": 1,
    "storageDriverName": "ontap-nas",
    "backendName": "nfs_project_3",
    "managementLIF": "172.21.224.210",
    "dataLIF": "10.61.181.228",
    "svm": "project-3-svm",
    "username": "vsadmin",
    "password": "NetApp!23"
}
EOF
```

이 예에서는 ONTAP-NAS 드라이버를 사용하고 있습니다. 사용 사례에 따라 백엔드를 생성하는 데 적합한 드라이버를 사용하십시오.

트리덴트 프로젝트에 Trident가 설치되어 있다고 가정합니다.

1. PROJECT-3의 스토리지 클래스를 생성하고 PROJECT-3 전용 백엔드에서 스토리지 풀을 사용하도록 구성합니다.

```
cat << EOF | oc create -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: project-3-sc
provisioner: csi.trident.netapp.io
parameters:
  backendType: ontap-nas
  storagePools: "nfs_project_3:.*"
EOF
```

2. ResourceQuota를 생성하여 다른 프로젝트 전용 스토리지로부터 스토리지를 요청하는 Project-3의 리소스를 제한합니다.

```
cat << EOF | oc create -f -
kind: ResourceQuota
apiVersion: v1
metadata:
  name: project-3-sc-rq
  namespace: project-3
spec:
  hard:
    project-1-sc.storageclass.storage.k8s.io/persistentvolumeclaims: 0
    project-2-sc.storageclass.storage.k8s.io/persistentvolumeclaims: 0
EOF
```

3. 다른 프로젝트의 ResourceQuotas를 패치하여 해당 프로젝트의 리소스가 project-3 전용 스토리지 클래스에서 스토리지에 액세스하는 것을 제한합니다.

```
oc patch resourcequotas project-1-sc-rq -n project-1 --patch
'{"spec":{"hard":{ "project-3-
sc.storageclass.storage.k8s.io/persistentvolumeclaims": 0}}}'
oc patch resourcequotas project-2-sc-rq -n project-2 --patch
'{"spec":{"hard":{ "project-3-
sc.storageclass.storage.k8s.io/persistentvolumeclaims": 0}}}'
```

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.